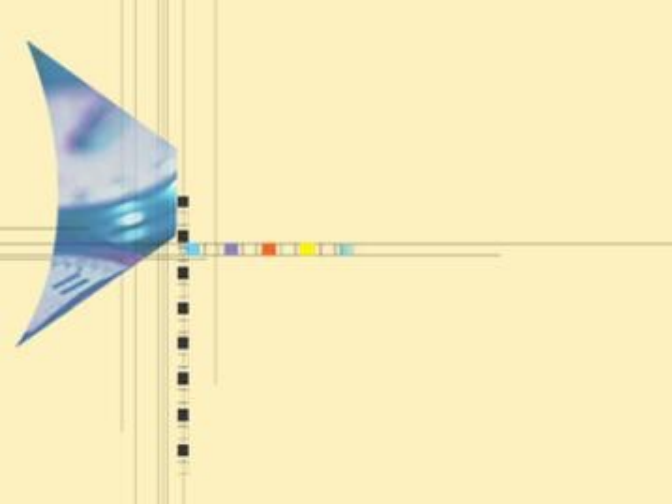




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

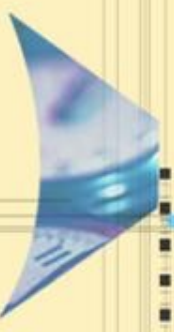
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/23

武汉大学计算机学院

武汉大学
Wuhan University





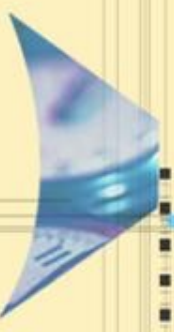
主要内容

9.1 引言

9.2 检索策略

9.3 分支限界算法设计思想

9.4 分支限界法的典型应用



主要内容

9.4. 分支限界法的典型应用（可选讲部分）

9.4.1 货郎担问题

9.4.2 作业排序问题

9.4.3 0/1背包问题

9.4.4 整数背包问题

9.4.5 最大团集问题

9.4.6 圆排列问题



9.4.1 TSP Problem (货郎担问题)

问题描述

- 给出一个城市的集合和一个定义在每一对城市间的耗费函数，找出耗费最小的游程。（最小化问题）
- 在这里，一个游程是一个闭合的路径，它访问每个城市恰好一次，也就是说，是一个简单回路。
- 耗费函数可以是距离、旅行时间、飞机票价等。
- 设TSP问题的一个含有 n 个城市的实例表示为一个 $n \times n$ 的耗费矩阵，矩阵中每个元素值为非负，代表两个城市间的旅行成本。

游程耗费的下界函数

- 给每个部分解 $(x_1, x_2, \dots, x_j)$ 一个相关联的下界 y ，其含义为：任何按这种次序访问了城市 $x_1, x_2, \dots, x_j$ 的完整游程的耗费必须至少是 y （即真实耗费的下界）。
- 如何计算 y ?

周游耗费的下界函数

观察结论:

- 每个完整的游程一定恰好包含来自耗费矩阵的每一行每一列的一条边和它关联的耗费。
- 如果耗费矩阵 A 的任意行和列的每个项减去常数 r 得到新矩阵, 那么在新矩阵下任何游程的耗费比 A 矩阵下同样游程的耗费恰好少 r ;
- 归约矩阵: 对原矩阵的相关行/列减去该行/列的最小值的, 使得新矩阵中每一行每一列至少有一个0, 则新矩阵为原矩阵的归约矩阵。被减去的总量称为矩阵约数。
- 由于归约矩阵下的任何游程的耗费至少为0, 因此原矩阵下相同的游程的耗费至少是减去的矩阵约数。
- 矩阵约数为原矩阵下任何完整游程的下界。

例子

- TSP问题的实例矩阵和它的归约矩阵

∞	17	7	35	18	→	∞	10	0	28	11	-7
9	∞	5	14	19	→	4	∞	0	0	14	-5
29	24	∞	30	12	→	17	12	∞	18	0	-12
27	21	25	∞	48	→	6	0	4	∞	27	-21
15	16	28	18	∞	→	0	1	13	0	∞	-15

-3

$$y=7+5+12+21+15+3=63$$

排列树形式组织解空间

- 解的表示：设有 n 个城市，周游路线从1开始，则解向量为 $\langle x_1, x_2, \dots, x_{n-1} \rangle$ ，其中 $x_1, x_2, \dots, x_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列。搜索空间为 $(n-1)!$ 对应的排列树。

子结点对应的游程耗费下界

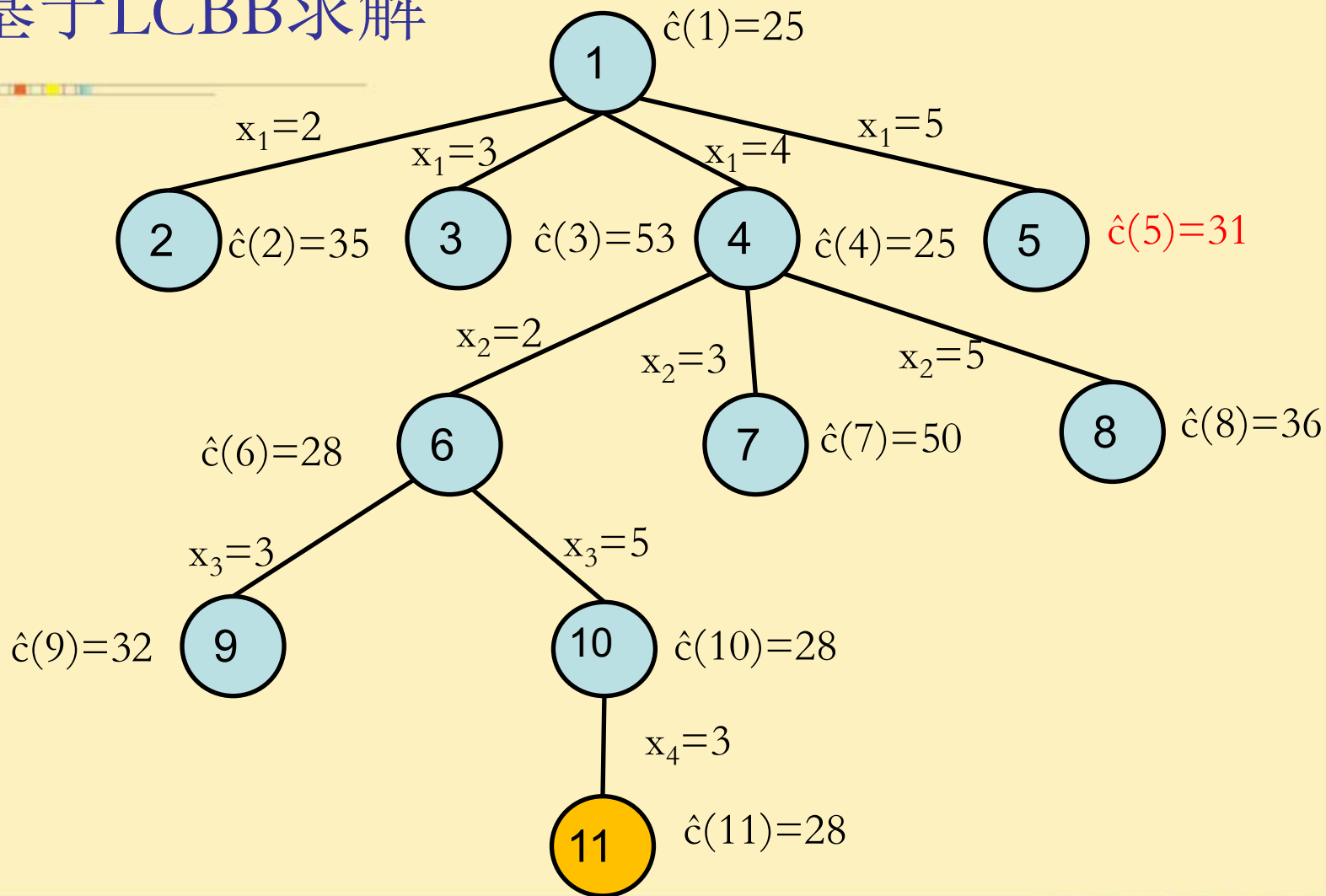
- 设A是结点R的归约矩阵，对应原矩阵下的游程下界为 $\hat{c}(R)$
- S是R的儿子，树边<R,S>对应游程中的边<i,j>。
- 在S是非叶结点的情况下，S的归约矩阵B按以下方法获得：
 1. 为保证这条游程采用边<i,j>而不采用其它由i出发或者进入j的边，将A中i行和j列的元素置为 ∞ ；
 2. 为防止使用边<j,i>，将A中j行i列的元素置为 ∞ ；
 3. 对于那些不全为 ∞ 的行和列施行归约规则得到S的归约成本矩阵，令其为B，矩阵约数r；
 4. 则S对应原矩阵下的游程下界 $\hat{c}(S)$ 可计算为：
$$\hat{c}(S) = \hat{c}(R) + A(i,j) + r$$
- 如果S是叶结点，由于一个叶子结点确定一条唯一的游程，因此 $\hat{c}(S) = c(S)$

例子

∞	20	30	10	11	→	∞	10	17	0	1	-10
15	∞	16	4	2	→	12	∞	11	2	0	-2
3	5	∞	2	4	→	0	3	∞	0	2	-2
19	6	18	∞	3	→	15	3	12	∞	0	-3
16	4	7	16	∞	→	11	0	0	12	∞	-4
						-1	-3				

$$y=10+2+2+3+4+1+3=25$$

基于LCBB求解



边的子集树形式组织解空间

- 解的表示：将一个游程看作是 n 条边的集合，从所有 m 条边中搜索构成最短游程的边的子集。因此，状态空间树为一棵二叉树，结点的左结点表示游程中包含一条指定的边，右子树表示游程中不包含指定的边。

左子结点对应的游程耗费的下界

- 设 A 是结点 R 的归约矩阵，对应原矩阵下的游程下界为 $\hat{c}(R)$
- S_L 是 R 的左儿子且对应 **包含边** $\langle i,j \rangle$ ， S_R 是 R 的右儿子且对应 **不包含边** $\langle i,j \rangle$ 。
- 在 S_L 是非叶结点的情况下， S_L 的归约矩阵 B_L 按以下方法获得：
 1. 为保证这条周游路线采用边 $\langle i,j \rangle$ 而不采用其它由 i 出发或者进入 j 的边，将 A 中 i 行和 j 列的元素置为 ∞ ；
 2. 为防止使用边 $\langle j,i \rangle$ ，将 A 中 j 行 i 列的元素置为 ∞ ；
 3. 对于那些不全为 ∞ 的行和列施行归约规则得到 S 的归约成本矩阵，令其为 B_L ，矩阵约数 r_L ；
- 则 S_L 对应原矩阵下的 **游程下界** $\hat{c}(S_L)$ 可计算为：
$$\hat{c}(S_L) = \hat{c}(R) + A(i,j) + r_L$$

右子结点对应的游程耗费的下界

- 设A是结点R的归约矩阵，对应原矩阵下的游程下界为 $\hat{c}(R)$
- S_L 是R的左儿子且对应包含边 $\langle i,j \rangle$ ， S_R 是R的右儿子且对应不包含边 $\langle i,j \rangle$ 。
- 由于 S_R 代表不包含边 $\langle i,j \rangle$ 的游程，因此应将R的归约成本矩阵A中的元素A(i,j)置成 ∞ 后，再归约此矩阵不全为 ∞ 的行和列（实际上只需重新归约第i行和第j列），即得 S_R 的归约成本矩阵 B_R 和矩阵约数： $r_R = \min_{k \neq j} \{A(i,k)\} + \min_{k \neq i} \{A(k,j)\}$
- 计算 S_R 对应原矩阵下的 $\hat{c}(S_R)$ 时，由于不包含边A(i,j)，因此不必加入，即：

$$\hat{c}(S_R) = \hat{c}(R) + r_R$$

边的取舍次序

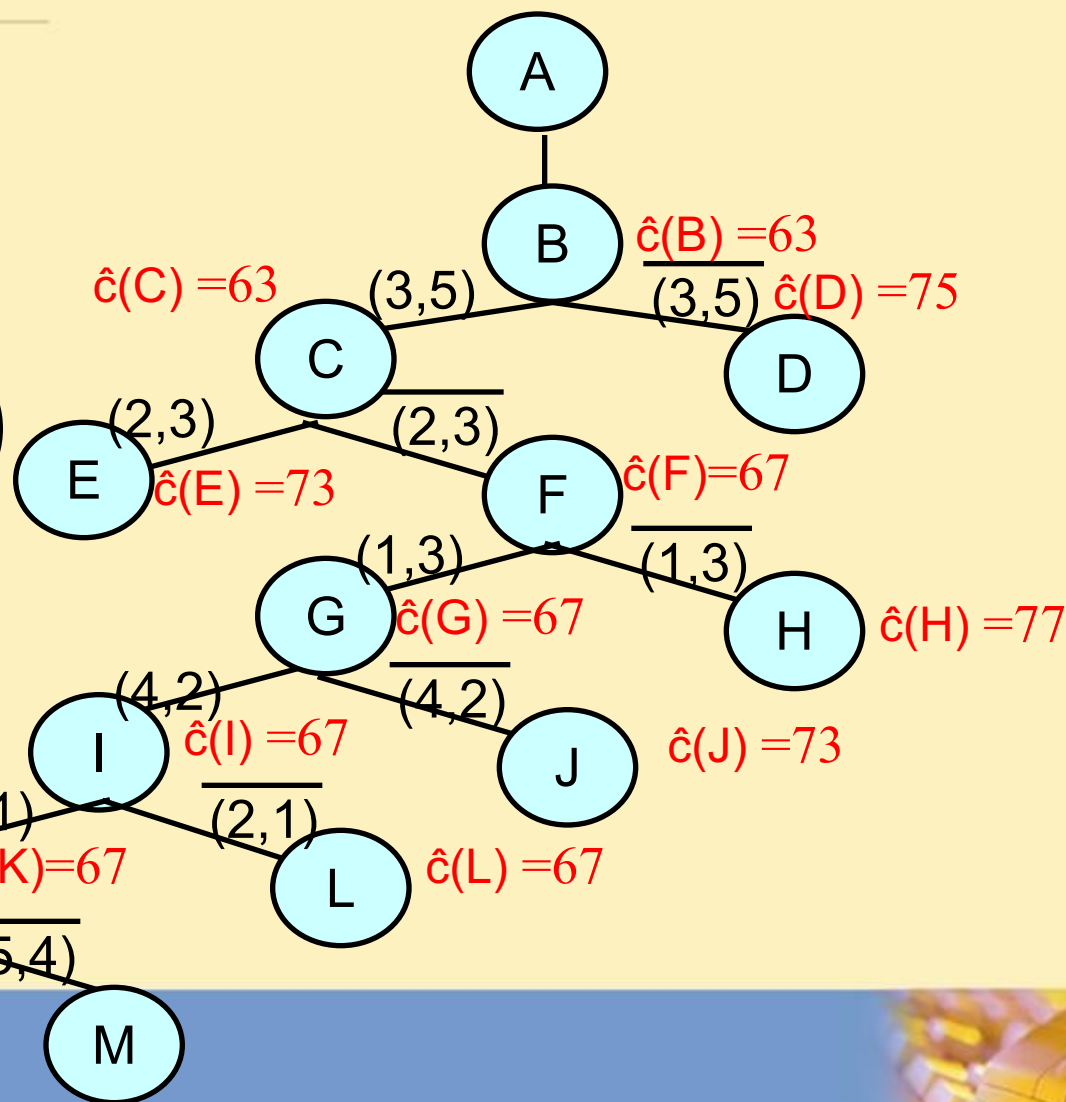
- 如果选取边 $\langle i, j \rangle$ ，则这条边将解空间分成两个子集合，根的左子树表示包含边 $\langle i, j \rangle$ 的所有游程，根的右子树表示不含边 $\langle i, j \rangle$ 的所有游程；
- 如果左子树中包含一条最小耗费游程，则只需再选取 $n-1$ 条边；如果所有的最小耗费游程都在右子树中，则还需要对边做 n 次选择；
- 在左子树中找最优解比在右子树中找最优解容易；
- 希望选取一条最有可能在最小耗费游程中的边 $\langle i, j \rangle$ 作为分割边。
- 选择边的规则：
 - 选取一条使其右子树具有最大 $\hat{c}$ 值的边，OR，
 - 选取使左、右子树 $\hat{c}$ 值相差最大的边。。。

选取一条使其右子树具有最大 $\hat{c}$ 值的边

- 如果选取的边 $\langle i, j \rangle$ 在 R 的归约矩阵 A 中对应的元素 $A(i, j)$ 不为0, 则必有 $r_R = 0$, 即 $\hat{c}(S_R) = \hat{c}(R)$; 显然不能使 R 的右子树的 $\hat{c}$ 值最大;
- 因此, 为了使 R 的右子树的 $\hat{c}$ 值最大, 应从 R 的归约矩阵元素为0的对应边中选取有最大 r_R 值的边。

基于LCBB求解

∞	2	7	35	1
9	∞	5	14	19
4	24	∞	30	12
27	21	5	∞	3
15	16	28	18	∞



SOLUTION!

讨论

- 最优解的初始上界：设定一个足够大的初始上界，之后根据找到的最好解不断修订。上例中，初始时无显式最优解上界，认为是 ∞ 。
- 更紧的初始上界：可以考虑由贪心策略确定。

9.4.2 作业排序问题

- n 个作业和1台处理机,每个作业 i 与一个三元组 (p_i, d_i, t_i) 联系, t_i 个单位处理时间,在截止期限 d_i 内没有处理完则要招致 p_i 的罚款.
- 目标: 从 n 个作业中选取一个子集合 J ,要求在 J 中的作业都能在相应的期限内完成且使不在 J 中作业招致的罚款总额最小。 (最小优化)
- 例如: $n=4$, $p=(5,10,6,3)$, $d=(1,3,2,1)$, $t=(1,2,1,1)$ 。解就是要穷尽4个作业的所有可能子集合确定。
- 设元组采用定长表示, 每个分量代表对应作用是否在子集合中。

9.4.2 作业排序问题

- 成本函数 $c(\cdot)$ 定义: 对于可行结点 X , $c(X)$ 是根为 X 的子树中结点的最小罚款; 对于不可行结点 X , $c(X) = \infty$.
- 估计成本函数 $\hat{c}(X)$ 的定义:

设 S_x 是在结点 X 对应选择的作业子集, 如果

$$m = \max \{i \mid i \in S_x\}, \text{ 则 } \hat{c}(X) = \sum_{(i < m, i \notin S_x)} p_i$$

- 显然, $\hat{c}(X) \leq c(X)$; 当 X 为答案节点时, $\hat{c}(X) = c(X)$

9.4.2 作业排序问题

- 结点X的简单上界定义: $u(X) = \sum_{(i \notin S_X)} p_i$
- 最小成本答案结点的成本上界U的设置方法:
 - 初值: ∞ 或 $\sum_{(1 \leq i \leq m)} p_i$
 - 运行过程中修订: 已生成的活结点的最小的 $u(X)$
- 杀死活结点策略: U不是成本值时, 杀死所有令 $\hat{c}(X) > U$ 的活结点; U是成本值时, 杀死所有令 $\hat{c}(X) \geq U$ 的活结点.

9.4.2 作业排序问题

- 由于 U 不断修正，队列中有些活结点可能不再满足条件需要杀死。
每次从活结点表中找出可杀活结点很不经济。算法实现时经济的处理方法：每当可杀结点要成为E-结点时才杀死。
- 统一杀死活结点策略：杀死所有令 $\hat{c}(X) \geq U$ 的活结点。为此，取足够小正常数 ε ，使得如果 $u(X) < u(Y)$ ，则 $u(X) < u(X) + \varepsilon < u(Y)$ 。
- U 是由一答案结点的成本值得到时， U 就取该成本值；如果 U 是由一单纯上界得到时， $U = u(X) + \varepsilon$

9.4.3 0/1背包问题

问题描述：有 n 个重量分别为 $\{w_1, w_2, \dots, w_n\}$ 的物品，它们的价值分别为 $\{v_1, v_2, \dots, v_n\}$ ，给定一个容量为 W 的背包。从这些物品中选取一部分物品放入该背包的方案，每个物品要么选中要么不选中，要求选中的物品不仅能够放到背包中，而且重量和不超过 W ，且具有最大的价值。

解向量：定长元组表示， $x = (x_1, x_2, \dots, x_n)$ 。

例：假设一个0/1背包问题是， $n=3$ ，重量为 $w = (16, 15, 15)$ ，价值为 $v = (45, 25, 25)$ ，背包限重为 $W=30$ 。解向量为 $x = (x_1, x_2, x_3)$ 。

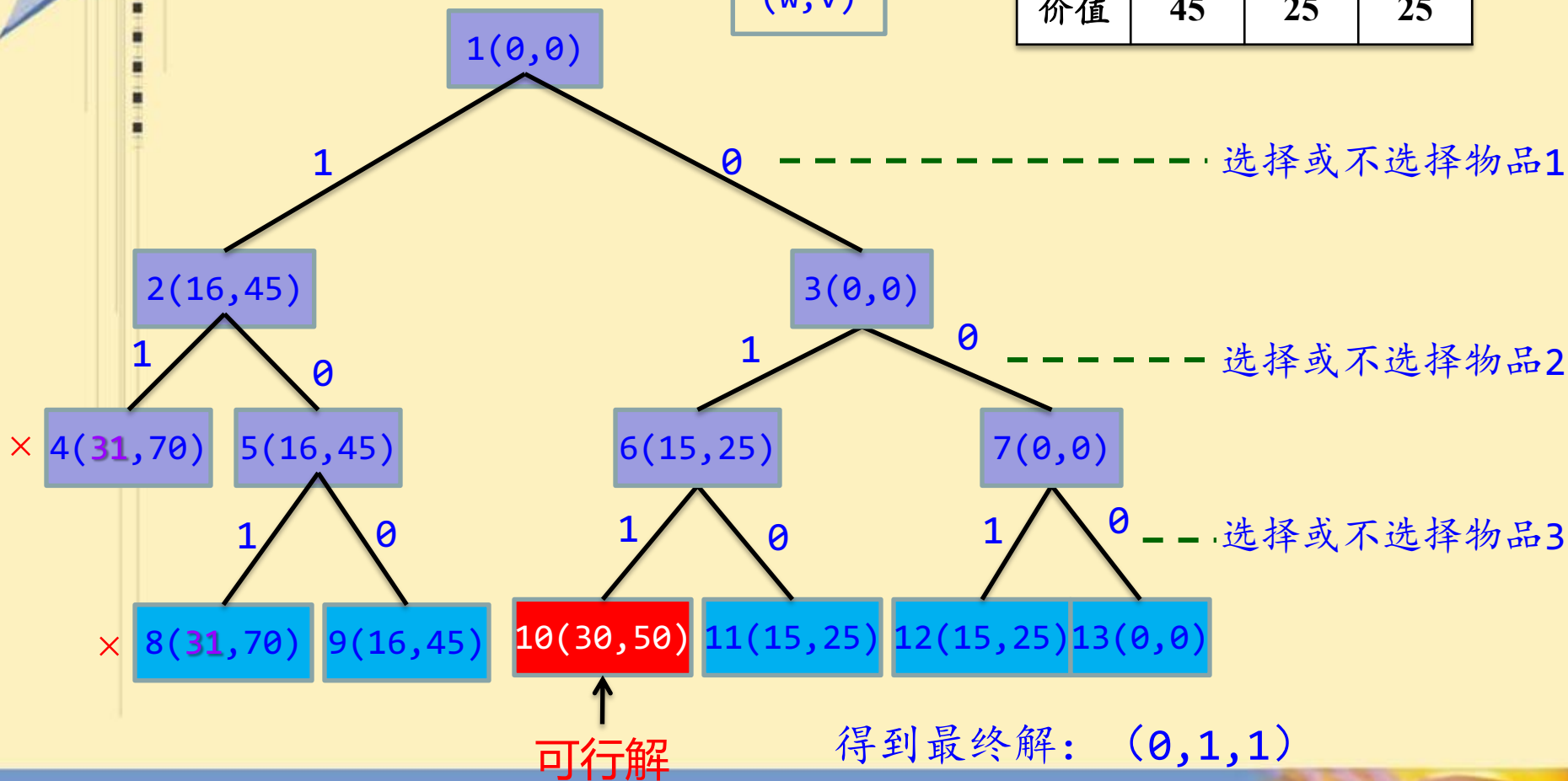
编号	1	2	3
重量	16	15	15
价值	45	25	25

队列式分支限界求解

首先不考虑限界问题 - FIFO

编号	1	2	3
重量	16	15	15
价值	45	25	25

(w, v)



队列式分支限界求解

考虑限界问题

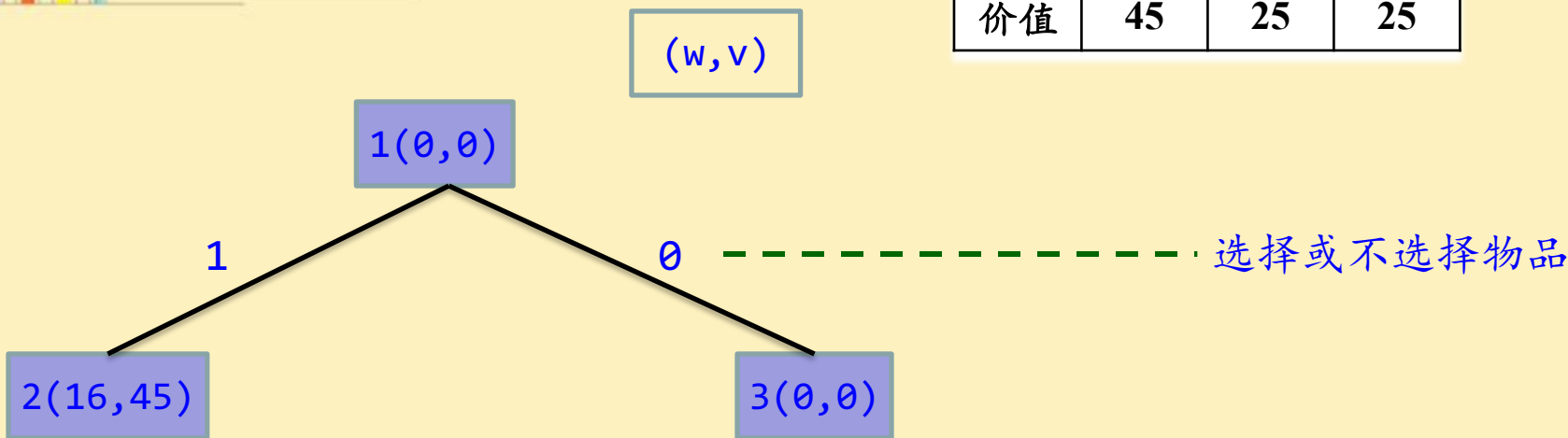
该问题是求装入背包的最大价值，属求**最大值**问题，直接采用求最大值的方法。因此对结点对应的价值上界进行设计。

回溯策略中限界函数一样

- 如果所有剩余的物品都能装入背包，那么价值的上界为当前装到背包中物品总价值+剩余所有物品总价值；
- 如果所有剩余的物品不能全部装入背包，那么对剩余物品采用分数背包问题的贪心法。得到价值的上界

物品已按单位重量价值降序排列

编号	1	2	3
重量	16	15	15
价值	45	25	25



根结点1 ($w=0, v=0$), 结点价值的上界:

$$\hat{c} = 0 + 45 + (30-16) \times 25/15 = 68 \text{ (取整)}$$

$w=0$

$w[1]=16 < 30$
可选物品1
 $v[1]=45$

可选物品2的一
部分, 即 $30-16$
, 对应的价值

利用限界值剪枝条件- FIFOBB

假设当前扩展结点扩展了两个子结点： $e \rightarrow e1, e2$ ，剪枝

- 左分支 $e1$ ：表示选择了物品 i 的分支，如果当前背包中物品总重量+物品 i 的重量 w_i ，没有超过背包容量，则保留成为活结点。也就是说，如果 $cw + w_i > W$ ，则剪枝。
- 右分支 $e2$ ：表示不选择物品 i 的分支，那么如果 $e2$ 结点的价值上界超过当前最好解的价值 $bestv$ ，则扩展 $e2$ 才有可能得到更好的解，因此保留成为活结点。也就是说，如果选择剩余所有满足限重物品都不能得到比目前找到的最好解更好的解，即 $\hat{c}(e2) \leq bestv$ ，就剪枝。

优先队列式分支限界求解

采用优先队列式分支限界法求解就是将一般的队列改为优先队列，但必须设计结点的上界函数 $\hat{c}()$ ，因为优先级是以限界函数值为基础的。

参照UCBB算法求解



算法分析

无论采用队列式分枝限界法还是优先队列式分枝限界法求解0/1背包问题，最坏情况下要搜索整个解空间树，所以最坏时间和空间复杂度均为 $O(2^n)$ ，其中 n 为物品个数。



9.4.4 整数背包问题

- 解的表示：定长元组 $(x_1, x_2, \dots, x_n)$, x_i 为非负整数
- 目标函数: $\max (\sum_{(1 \leq i \leq n)} v_i x_i)$
- 约束条件(可行性): $\sum_{(1 \leq i \leq n)} w_i x_i \leq b$
- 结点的效益函数的定义:
 - 对每一个答案结点 X , $c(X) = \sum_{(1 \leq i \leq n)} v_i x_i$;
 - 对不可行的叶结点, $c(X) = -\infty$;
 - 对于非叶结点, $c(X) = \max \{c(\text{LCHILD}(X)), c(\text{RCHILD}(X))\}$

9.4.4 整数背包问题

- 设物品按单位重量价值降序排列。如剩余容量能放下 $k+1$ 或者以后某个物品时，上界为当前背包物品中价值加上剩余容量能达到的最大价值；否则为当前背包价值。
- 因此，结点 $(x_1, x_2, \dots, x_k)$ 最大效益值的上界: UB (.)

$$\sum_{i=1}^k v_i x_i + (b - \sum_{i=1}^k w_i x_i) \frac{v_{k+1}}{w_{k+1}}$$

$$\text{若对某个 } j > k \text{ 有 } b - \sum_{i=1}^k w_i x_i \geq w_j$$

$$\sum_{i=1}^k v_i x_i$$

否则

9.4.4 整数背包问题

- 因此，估计效益函数（上界） $\hat{c}(\cdot)$ 的定义: UB ($\cdot$)
- 定义当前结点的最大效益值的下界函数 $lo(\cdot)$ ，即为当前背包中物品效益值
- 显然，对每个可行结点 X : $lo(X) \leq c(X) \leq \hat{c}(X)$
- 最优解的下界：初始值为0。随着搜索过程更新，若结点可能得到的背包最大价值大于目前最优解的下界，则更新最优解的下界。

实例

- 设 $n=4$, 按单位重量价值降序排列

$$\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$$

- $v=(9,5,3,1)$, $w=(7,4,3,2)$, $b=10$

- 求 $\max(\sum_{i=1}^4 v_i x_i)$

$$\text{s.t. } \sum_{i=1}^4 w_i x_i \leq 10$$

$$x_i \in \mathbb{N},$$

$$0 \leq x_1 \leq 1, 0 \leq x_2 \leq 2,$$

$$0 \leq x_3 \leq 3, 0 \leq x_4 \leq 5$$

回溯搜索

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10, \quad x_i \in \mathbb{N}, i = 1, 2, 3, 4$$

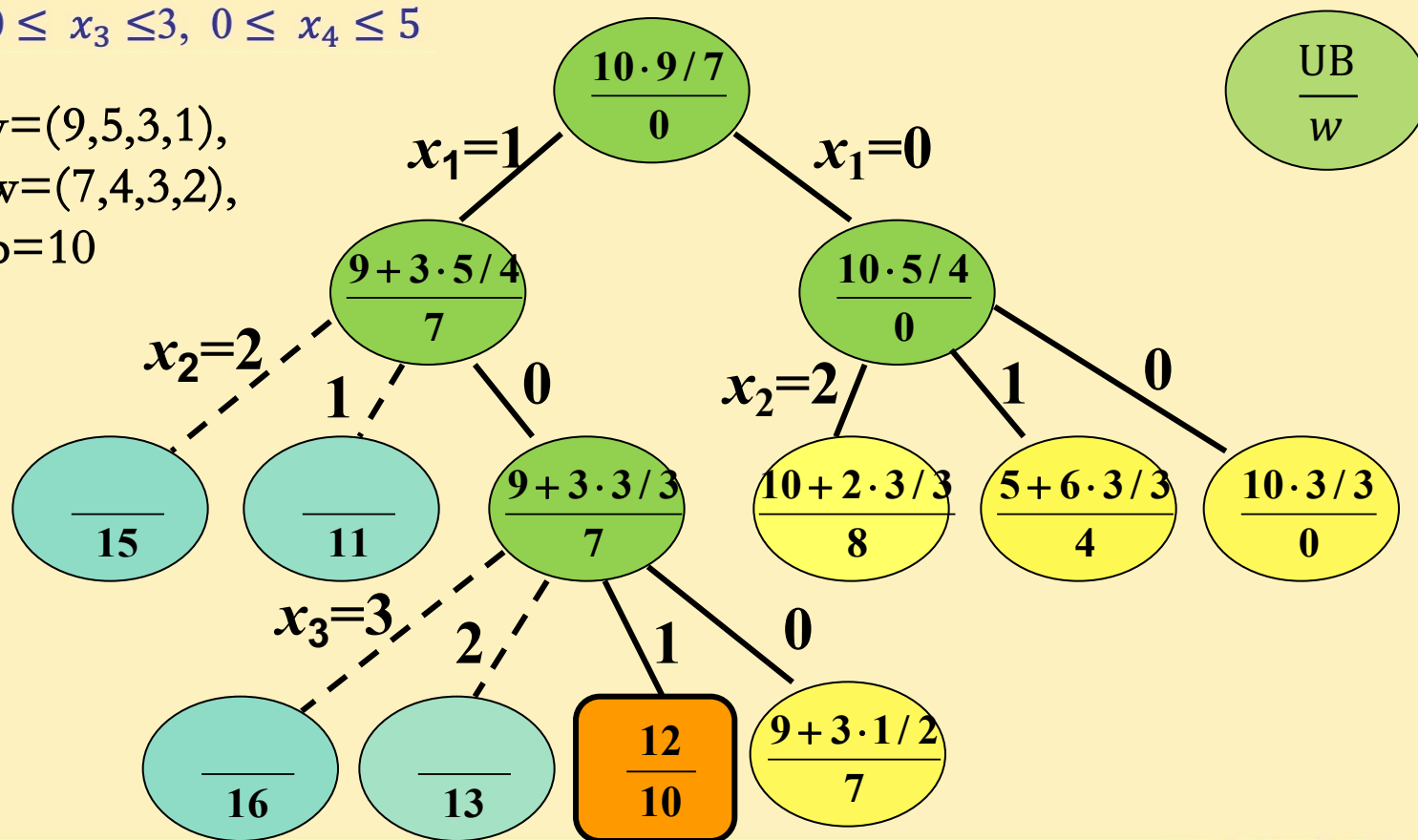
$$0 \leq x_1 \leq 1, 0 \leq x_2 \leq 2,$$

$$0 \leq x_3 \leq 3, 0 \leq x_4 \leq 5$$

$$v = (9, 5, 3, 1),$$

$$w = (7, 4, 3, 2),$$

$$b = 10$$



FIFOBB

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10, \quad x_i \in \mathbb{N}, i = 1, 2, 3, 4$$

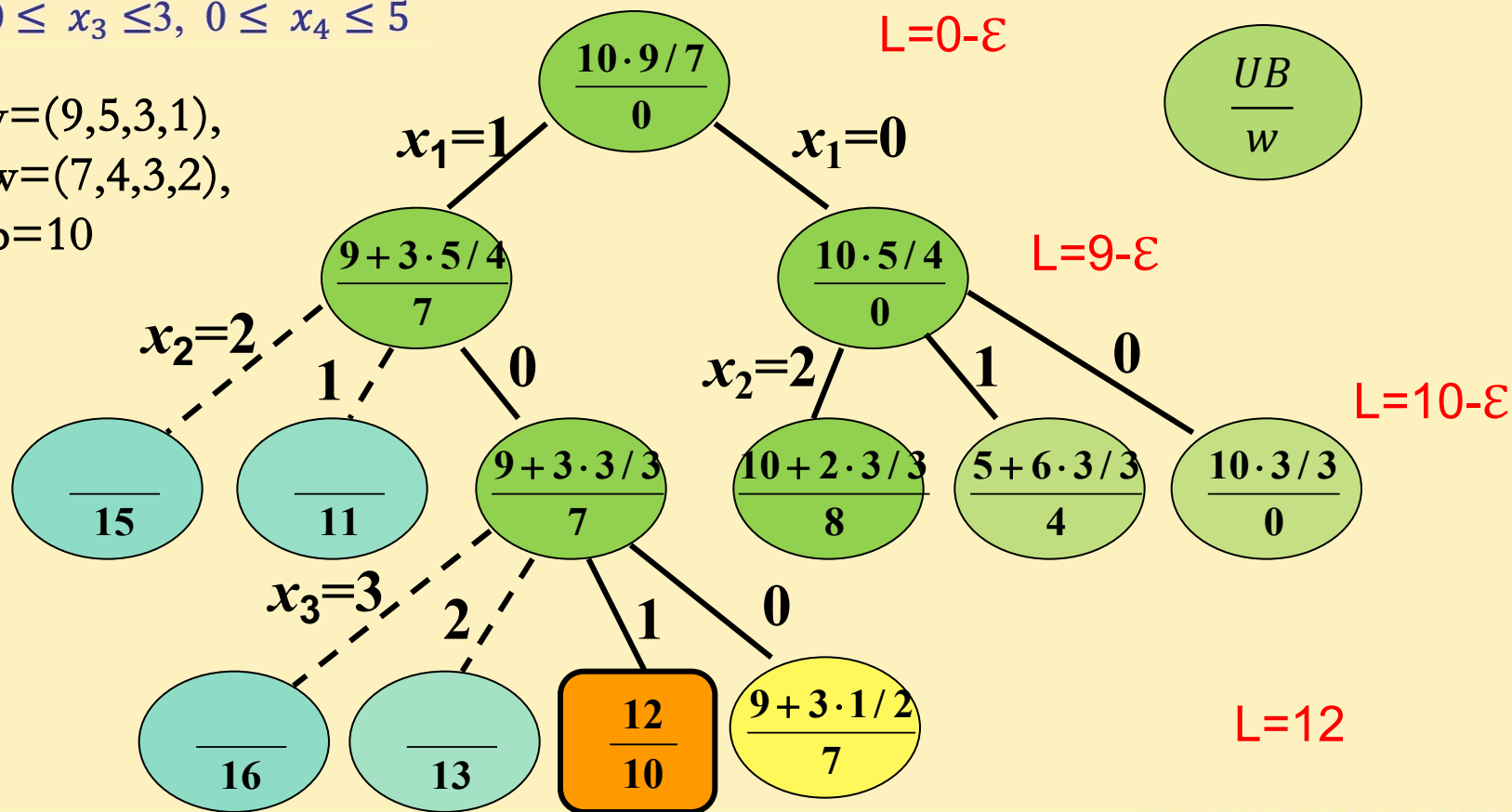
$$0 \leq x_1 \leq 1, 0 \leq x_2 \leq 2,$$

$$0 \leq x_3 \leq 3, 0 \leq x_4 \leq 5$$

$$v = (9, 5, 3, 1),$$

$$w = (7, 4, 3, 2),$$

$$b = 10$$



UCBB

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10, \quad x_i \in \mathbb{N}, i = 1, 2, 3, 4$$

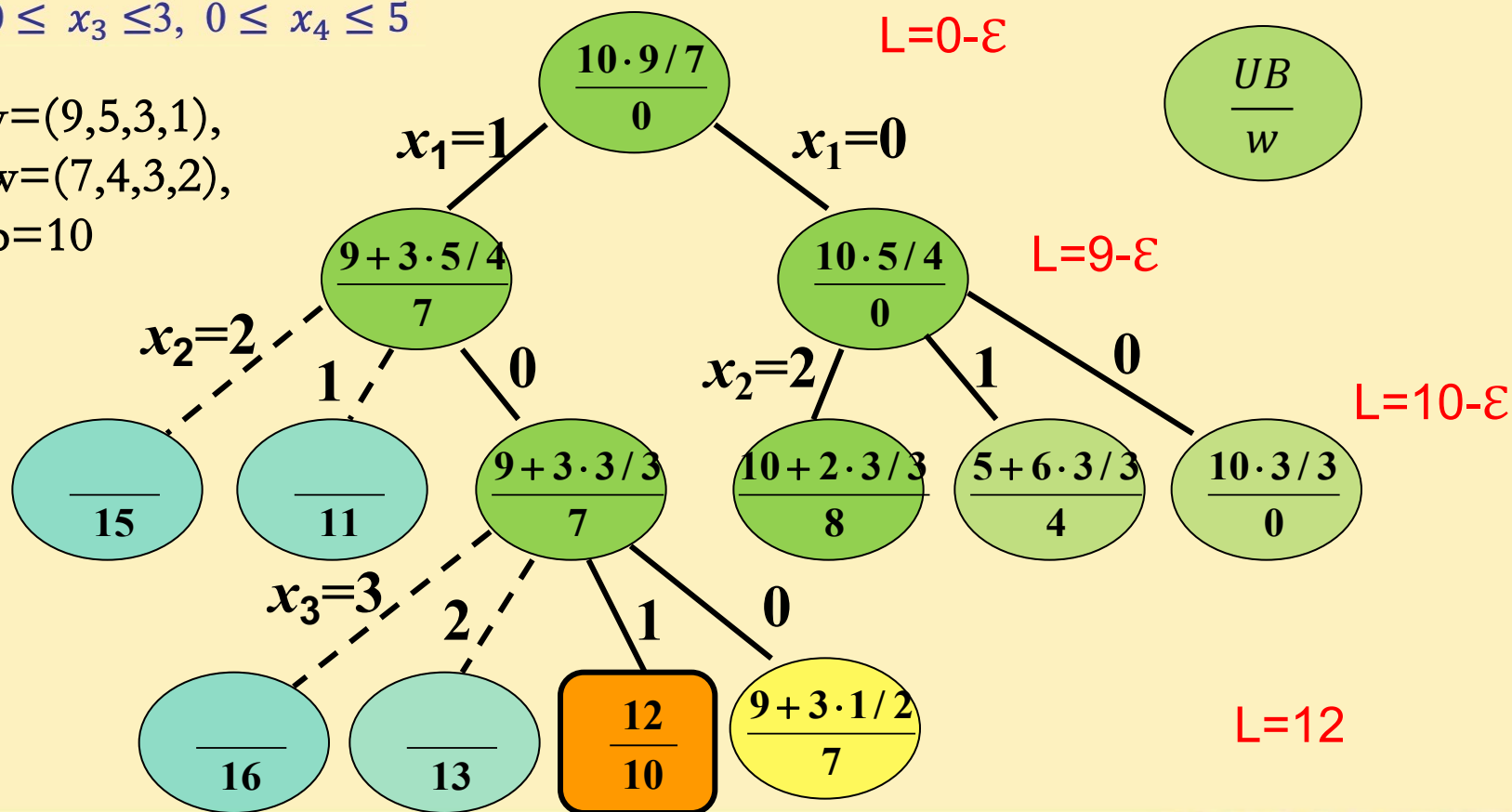
$$0 \leq x_1 \leq 1, 0 \leq x_2 \leq 2,$$

$$0 \leq x_3 \leq 3, 0 \leq x_4 \leq 5$$

$$v = (9, 5, 3, 1),$$

$$w = (7, 4, 3, 2),$$

$$b = 10$$



9.4.5 最大团集问题

最大团集问题：给定无向图 $G = \langle V, E \rangle$, 求 G 中的最大团集.

相关知识：

G 中的团： G 的完全子图

最大团：顶点数最多的团



算法关键部分设计

解的表示: n 元组 $(x_1, x_2, \dots, x_n)$, 其中 $x_i = 1$ 表示对应的顶点在当前的团内, $x_i = 0$ 表示不在。结点 $\langle x_1, x_2, \dots, x_k \rangle$ 表示已检索 k 个顶点。

解空间组织: 子集树

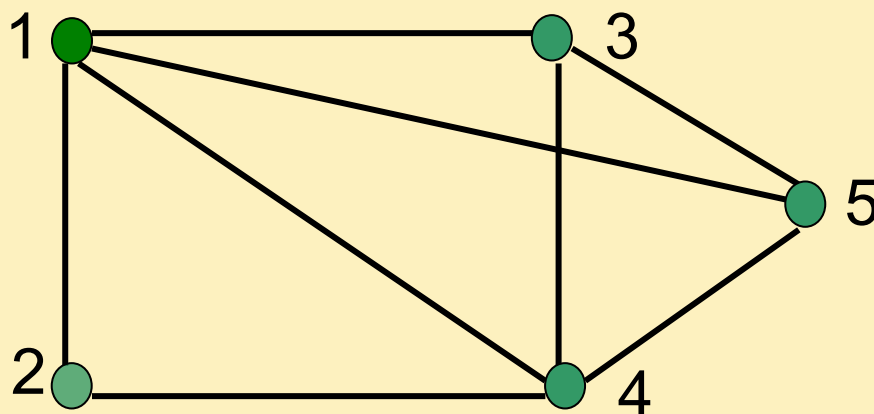
约束条件: 该分支结点对应顶点与当前团内每个顶点都有边相连

最优解的下界B: 当前图中已检索到的极大团的顶点数

结点的代价函数 $F()$: 目前的团扩张为极大团的顶点数上界。设其中 C_n 为目前团的顶点数 (初始为 0), 当前结点 $\langle x_1, x_2, \dots, x_k \rangle$, 即已检索 k 个顶点, 则剩下未检索顶点数为 $n-k$ 。因此, $F() = C_n + n - k$

最坏情况时间: $O(n2^n)$

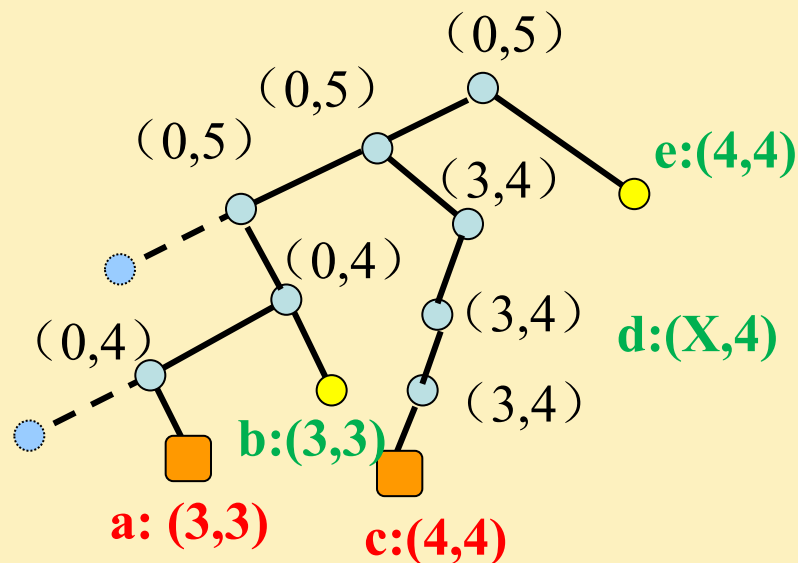
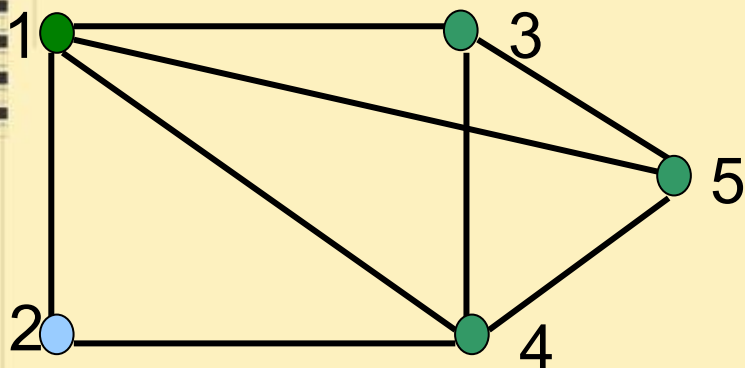
实例



顶点编号顺序为 1, 2, 3, 4, 5

对应 x_1, x_2, x_3, x_4, x_5 , $x_i=1$ 当且仅当 i 在团内
分支规定左子树为 1, 右子树为 0.

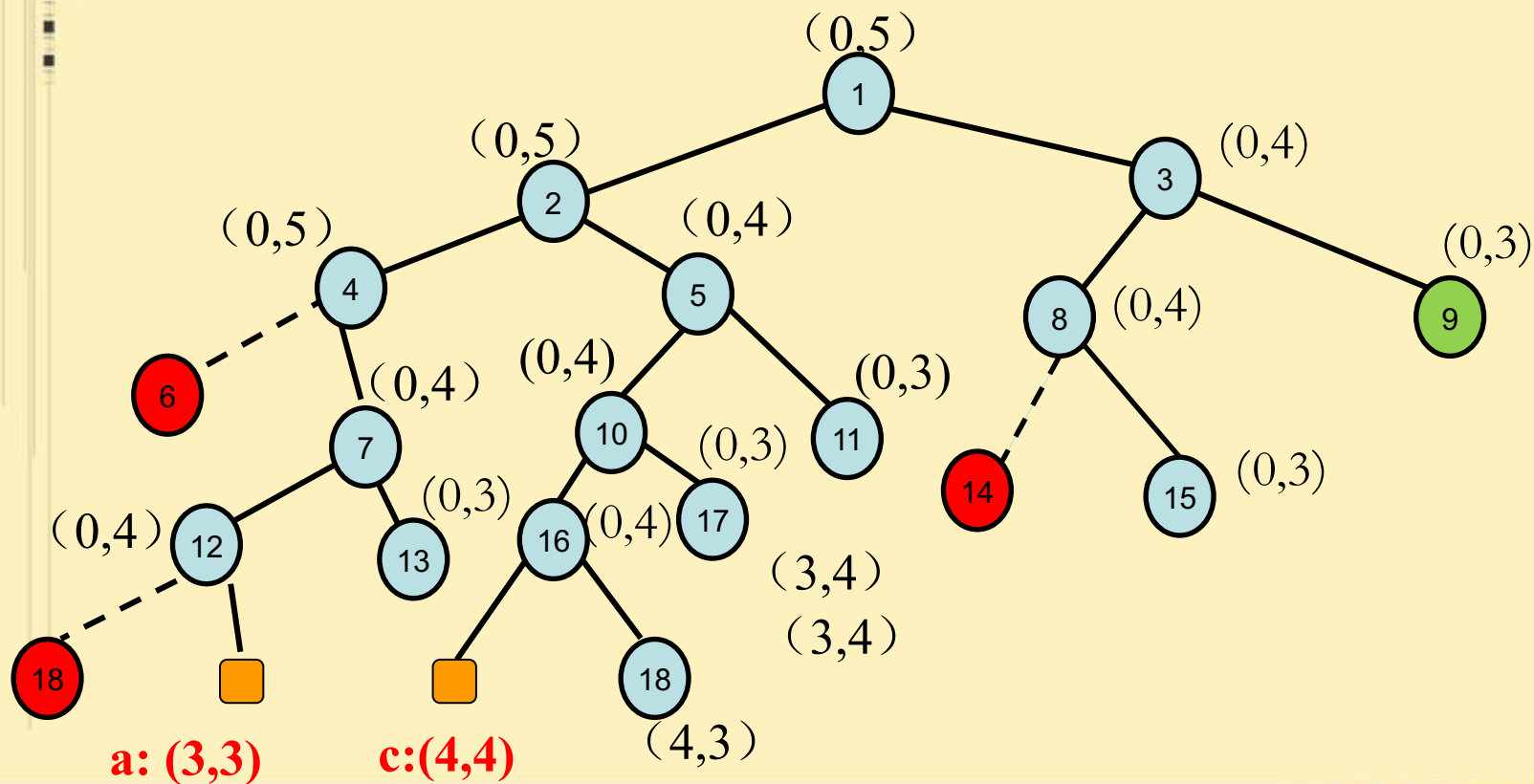
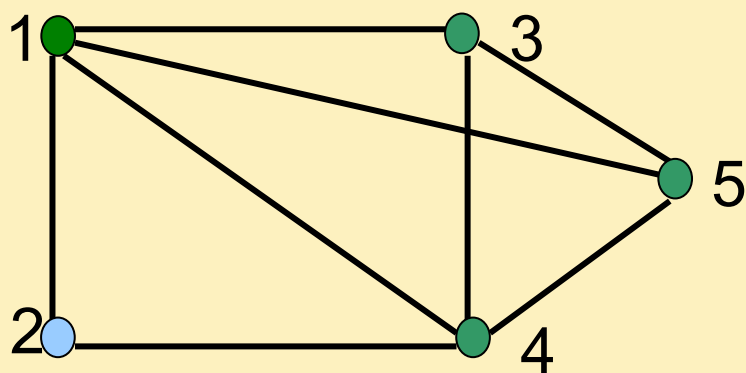
B 为最优解的界, F 为结点的代价函数值.

$$(B, F)$$


- a: 得第一个极大团 $\{1, 2, 4\}$, 顶点数为 3, 界为 3;
- b: 代价函数值 $F=3$, 回溯;
- c: 得第二个极大团 $\{1, 3, 4, 5\}$, 顶点数为 4, 修改界为 4;
- d: 不必搜索其它分支, 因为 $F=4$, 不超过界;
- e: $F=4$, 不必搜索.
- 最大团为 $\{1, 3, 4, 5\}$, 顶点数为 4.

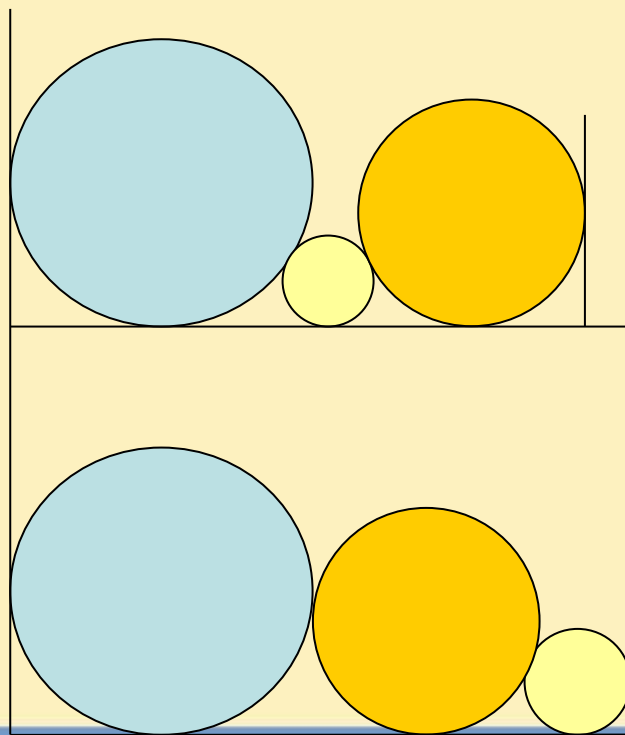
UCBB

(B, F)



9.4.6 圆排列问题

圆排列问题：给定 n 个圆的半径序列，将各圆与矩形底边相切排列，求具有最小长度 l_n 的圆的排列顺序。



9.4.6 圆排列问题

解的表示：解为 $\langle i_1, i_2, \dots, i_n \rangle$ 为 $1, 2, \dots, n$ 的排列，解空间为排列树。

部分解向量 $\langle i_1, i_2, \dots, i_k \rangle$ ：表示前 k 个圆已排好。

约束条件：令 $B = \{i_1, i_2, \dots, i_k\}$ ，下一个圆选择 i_{k+1} ，

$$i_{k+1} \in \{1, 2, \dots, n\} - B$$

最优解的上界：当前得到的最小圆排列长度，初值 ∞



结点的代价函数设计

k : 算法执行第 k 步, 已经选择了第 1— k 个圆

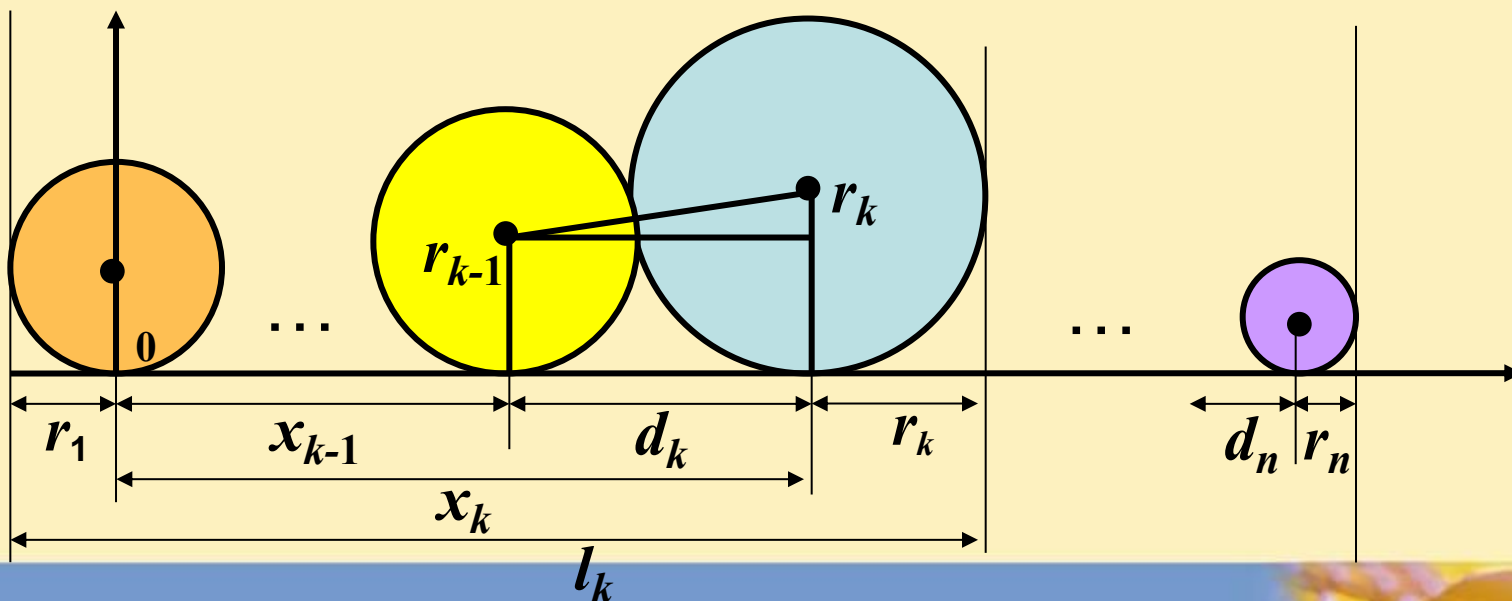
r_k : 第 k 个圆的半径

d_k : 第 $k-1$ 个圆到第 k 个圆的圆心水平距离, $k > 1$

x_k : 第 k 个圆的圆心横坐标, 规定 $x_1 = 0$,

l_k : 前 k 个圆的排列长度

L_k : 放好 k 个圆以后, 对应结点的代价函数值 $L_k \leq l_n$



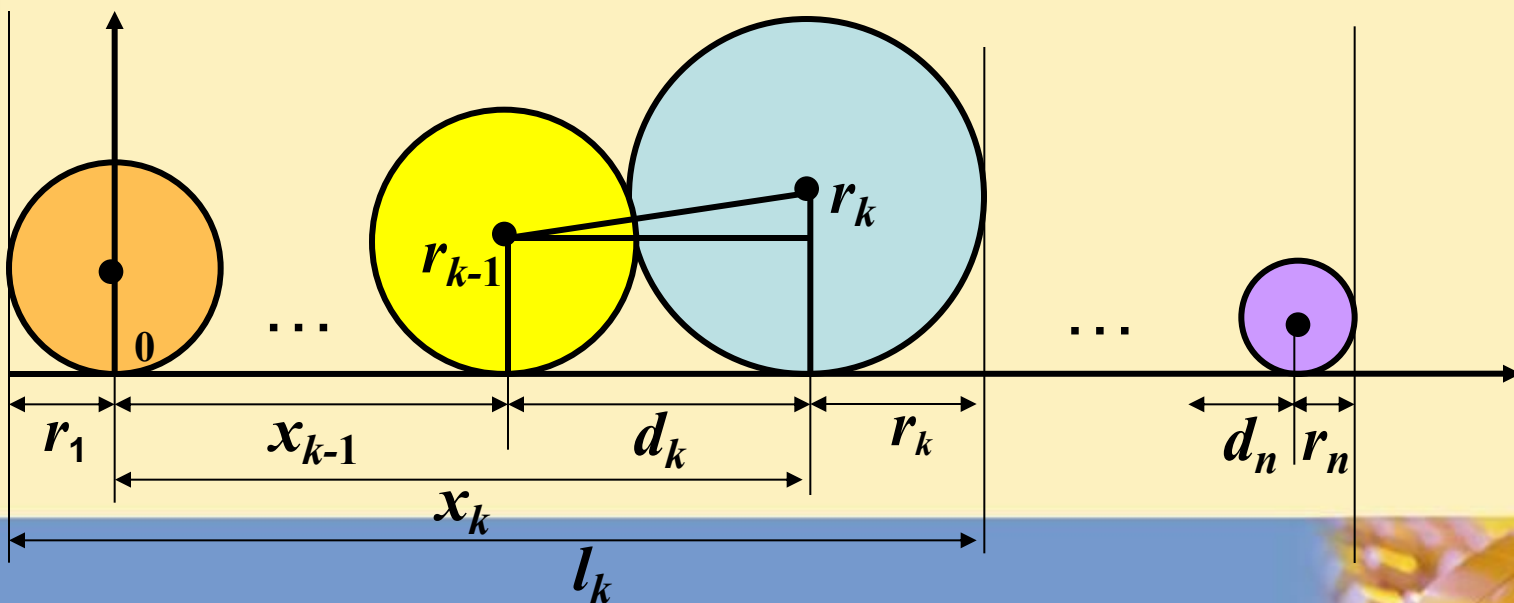
结点的代价函数设计

$$d_k = \sqrt{(r_{k-1} + r_k)^2 - (r_{k-1} - r_k)^2} = 2\sqrt{r_{k-1}r_k}$$

$$x_k = x_{k-1} + d_k, \quad l_k = x_k + r_k + r_1$$

$$L_k = x_k + d_{k+1} + d_{k+2} + \dots + d_n + r_n + r_1$$

$$= x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1$$



结点的代价函数设计

排列长度是 l_n , L 是代价函数:

$$l_n = x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1$$

$$\geq x_k + 2(n-k)r + r + r_1$$

$$L = x_k + (2n - 2k + 1)r + r_1$$

$$r = \min(r_{i_j}, r_k) \quad i_j \in \{1, 2, \dots, n\} - B$$

$$B = \{i_1, i_2, \dots, i_k\},$$

时间: $O(n n!) = O((n+1)!)$

实例

6个圆半径: $R = \{ 1, 1, 2, 2, 3, 5 \}$

设排列 $\langle 1, 2, 3, 4, 5, 6 \rangle$,

半径排列为: 1, 1, 2, 2, 3, 5, 结果见下表和下图

k	r_k	d_k	x_k	l_k	L_k
1	1	0	0	2	12
2	1	2	2	4	12
3	2	2.8	4.8	7.8	19.8
4	2	4	8.8	11.8	19.8
5	3	4.9	13.7	17.7	23.7
6	5	7.7	21.4	27.4	27.4